Penetration Testing Report

Full Name: Egeonu Chioma Lilian
Program: HCS
Date:17/02/2025

Introduction

This report document hereby describes the proceedings and results of a HTML Injection and Clickjacking conducted against the **Week 1 Labs**. The report hereby lists the findings and corresponding best practice mitigation actions and recommendations.

1. Objective

The aim of this study is to investigate and analyze HTML injection and Cross Site Scripting vulnerabilities as demonstrated in **Week 1 Labs**. The objective of this assessment is to identify, document, and understand these vulnerabilities thoroughly, outlining remediation strategies and providing recommendations to mitigate the risks associated with HTML injection and Cross Site Scripting in web applications.

2. Scope

This section defines the scope and boundaries of the project.

Application Name	**HTML Injection, Cross Site scripting**

3. Summary

Outlined is a Black Box Application Security assessment for the **Week 1 Labs**.

Total number of Sub-labs: 8 Sub-labs

High	Medium	Low
5	3	8

High - **Number of Sub-labs with hard difficulty level**

Medium - **Number of Sub-labs with Medium difficulty level**

Low - **Number of Sub-labs with Easy difficulty level**

1. HTML Injection

1.1. HTML's Are Easy!

Reference	Risk Rating
HTML's Are Easy!	**Low**
Tools Used	
Exploiting a straightforward HTML injection method via the search bar functionality (No tool used).	
Vulnerability Description	
The vulnerability allows an attacker to insert a malicious HTML code into the search bar of the website. When the search query containing the injected code is submitted, the website's server processes it and responds with a page that includes the injected code. This can lead to data breaches and website defacement.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/html_lab/lab_1/html_injection_1.php	
Consequences of not Fixing the Issue	
the impact of not fixing this vulnerability includes data breaches, website defacement, malware distribution, cookies theft, manipulation of page content, redirection to malicious websites, or other unauthorized actions.	
Suggested Countermeasures	
Robust input validation and sanitization procedures to mitigate the risk of HTML injection attacks, regular security audits.	
References	
https://www.imperva.com/learn/application-security/html-injection/ https://anithaana3.medium.com/easy-ways-to-exploit-html-injection-d06a594b9577	

Proof of Concept

1.2. Let Me Store Them!

Reference	Risk Rating
Let Me Store Them!	**Low**
Tools Used	
Manual analysis without specific tools.	
Vulnerability Description	
Stored HTML injection attack occurs when the malicious code is permanently stored on the target's server which is then served to the users whenever they visit a particular website and persistent cross site scripting attacks.	
How It Was Discovered	
Through manual analysis of input fields and subsequent observation of injected HTML code.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/html_lab/lab_2/html_injection_2.php	
Consequences of not Fixing the Issue	
Risk of data theft, unauthorized access, reputational damage, and legal liabilities and website defacement.	
Suggested Countermeasures	
Implement strict input validation, output encoding, and regular security audits.	
References	
https://portswigger.net/web-security/file-upload https://www.imperva.com/learn/application-security/html-injection/	

Proof of Concept

1.3. File Names are also vulnerable!

Reference	Risk Rating
File Names are also vulnerable!	Low
Tools Used	
Manual	
Vulnerability Description	
This HTML injection vulnerability is a case where a website allows users to upload files which includes malicious scripts and codes without proper validation. This could lead to codes been executed upon upload.	
How It Was Discovered	
Through manual analysis and page inspection..	
Vulnerable URLs	
https://labs.hacktify.in/HTML/html_lab/lab_3/html_injection_3.php	
Consequences of not Fixing the Issue	
Risk of XSS attacks, data manipulation, and unauthorized access to sensitive information, leading to compromised security, reputation damage, and legal liabilities.	
Suggested Countermeasures	
Implement robust input validation and sanitization, implement content security policies(CSP) to mitigate HTML injection vulnerabilities.	
References	
https://portswigger.net/burp/documentation/desktop/getting-started/modifying-http-requests https://hackerone.com/reports/331110	

Proof of Concept

```
    <center>
      <div class="containers">
              <h1>Upload a File</h1><br />
              <div class="welcome-image">
   <form action="html_injection_4.php" method="POST" enctype="multipart/form-data">
                      <input type="file" name="image"><br><br>
                      <button type="submit" name="upload" class="btn btn-warning">File Upload</button>
                  </form>
              </div>
              <h4>This fie contains some text.this is the text.</h4>
lnerable to <b>HTML injection</b>.</h3><center><br>File Uploaded Named <b>file.txt</b></center>
      </div>
      </center>
  </div>
<header>
div>
section>
r />
<div class="footer">
    <p>© Copyrights 2021 Hacktify Cybersecurity All rights reserved</p>
</div>
div>
>
>
```

1.4. File Content and HTML Injection a perfect pair!

Reference	Risk Rating
File Content and HTML Injection a perfect pair!	**Medium**
Tools Used	
Manual analysis.	
Vulnerability Description	
The vulnerability allows for HTML injection through file upload functionality, enabling the insertion of malicious code into the web page.	
How It Was Discovered	
Discovered through manual analysis of the file upload functionality.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/html_lab/lab_4/html_injection_4.php	
Consequences of not Fixing the Issue	
Failure to patch the vulnerability could result in malicious code execution, data theft, compromise of user confidentiality and persistent Cross Site Scripting	
Suggested Countermeasures	
Implement strict input validation, file type verification, and output encoding to prevent HTML injection via file uploads.	
References	
https://owasp.org/Top10/A03_2021-Injection/	

Proof of Concept

1.5. Injecting HTML using URL

Reference	Risk Rating
Injecting HTML using URL	**Medium**
Tools Used	
manual.	

Vulnerability Description
Exploiting lack of input validation to inject HTML code, resulting in arbitrary code execution.
How It Was Discovered
Manual analysis by altering the URL parameters.
Vulnerable URLs
https://labs.hacktify.in/HTML/html_lab/lab_5/html_injection_5.php
Consequences of not Fixing the Issue
Risk of unauthorized content injection, XSS attacks, data compromise, and reputational damage.
Suggested Countermeasures
Implement robust input validation, output encoding, and regular security updates.
References
https://www.w3schools.com/tags/tag_img.asp (Image HTML tag reference code) Web_Application_Security_Testing/11-Client-side_Testing/03-Testing_for_HTML_Injection

Proof of Concept

1.6. Encode IT!

Reference	Risk Rating
Encode IT!	**High**
Tools Used	
Manual analysis and Online Encoder tool.	
Vulnerability Description	
HTML injection vulnerability exploited by converting HTML tags to hex code to bypass input validation.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	

https://labs.hacktify.in/HTML/html_lab/lab_6/html_injection_6.php
Consequences of not Fixing the Issue
Risk of unauthorized data manipulation, XSS attacks, and reputational damage if left unpatched.
Suggested Countermeasures
Implement robust input validation and output strong encoding to prevent HTML injection attacks, regularly update security measures and implement Content Security Policy(CSP)
References
https://www.w3schools.com/tags/ref_urlencode.ASP

Proof of Concept

2. Cross Site Scripting

2.1. Let's Do IT!

Reference	Risk Rating
Let's Hijack!	**Low**
Tools Used	
Manual analysis.	
Vulnerability Description	
This is a type of security vulnerability that can be found in some web applications. XSS attacks enable attackers to inject client-side scripts into web pages viewed by other users. A cross-site scripting vulnerability may be used by attackers to bypass access controls such as the same-origin policy	
How It Was Discovered	
Manual analysis during lab exercise.	
Vulnerable URLs	
Consequences of not Fixing the Issue	

Attackers can exploit this vulnerability which include untrusted data from HTTP headers in their responses, Data leak and theft and complete account takeover.
Suggested Countermeasures
Sanitize data before including it in HTTP request, input validation, do not allow users to post content without proper inspection and processing.
References
https://portswigger.net/web-security/cross-site-scripting https://www.fortinet.com/resources/cyberglossary/cross-site-scripting

Proof of Concept

2.2. File Upload(file name)

Reference	Risk Rating
File upload	**Medium**
Tools Used	
Manual analysis, Command line Interface(Kali)	
Vulnerability Description	
This type of Cross-site Scripting via File Upload, enables attackers to upload a file that contains cross-site scripting payload. This allows several different attack opportunities, mostly hijacking the current session of the user or changing the look of the page by changing the HTML on the fly to steal the user's credentials. This happens because the input entered by the user has been interpreted as HTML/JavaScript/VBScript by the browser.	
How It Was Discovered	
The vulnerability was identified through manual analysis of the web page source and page inspection.	
Vulnerable URLs	

https://labs.hacktify.in/HTML/clickjacking_lab/lab_2/testfileupload.php
Consequences of not Fixing the Issue
Hijacking user's active session, Changing the look of the page within the victim's browser, Mounting a successful phishing attack and Intercepting data and performing man-in-the-middle attacks.
Suggested Countermeasures
Implement a strong CSP as a defence-indepth measure, all input and output from the site or application should be filtered
References
https://www.invicti.com/web-vulnerability-scanner/vulnerabilities/cross-site-scripting-via-file-upload/

Proof of Concept

uj

Proof of Concept

2. Cross Site Scripting

2.3 file upload(file content)

Reference	Risk Rating
File upload!	**Hard**

Tools Used
Manual analysis and command line interface
Vulnerability Description
This type of Cross-site Scripting via File Upload, enables attackers to upload a file that contains cross-site scripting payload. This allows several different attack opportunities, mostly hijacking the current session of the user or changing the look of the page by changing the HTML on the fly to steal the user's credentials. This happens because the input entered by the user has been interpreted as HTML/JavaScript/VBScript by the browser.
How It Was Discovered
The vulnerability was identified through manual analysis of the web page source and page inspection.
Vulnerable URLs
Consequences of not Fixing the Issue
Attackers can exploit this vulnerability which include untrusted data from HTTP headers in their responses, Data leak and theft and complete account takeover.
Suggested Countermeasures
Sanitize data before including it in HTTP request, input validation, do not allow users to post content without proper inspection and processing.
References
https://portswigger.net/web-security/cross-site-scripting https://www.fortinet.com/resources/cyberglossary/cross-site-scripting

Proof of Concept

2.2. DOM!

Reference	Risk Rating
DOM!	**Hard**
Tools Used	
Manual	
Vulnerability Description	
DOM-based XSS vulnerabilities usually arise when JavaScript takes data from an attacker-controllable source, such as the URL, and passes it to a sink that supports dynamic code execution. This enables attackers to execute malicious JavaScript, which typically allows them to hijack other users' accounts	
How It Was Discovered	
The vulnerability was identified through manual analysis of the web page source and page inspection.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/clickjacking_lab/lab_2/testDom.php	
Consequences of not Fixing the Issue	
Link redirection through a phishing link, which can result to millions of users potentially have their credentials stolen and blame your web application, reputational image, legal liabilities	
Suggested Countermeasures	
Regular audit and application scan, proper user input processing and investigation	
References	
https://www.invicti.com/learn/dom-based-cross-site-scripting-dom-xss/	

Proof of Concept